

Cross-Cert Synergy Pack — Four Claude Exams in Four Days

Contents

Cross-Cert Synergy Pack — study once, deploy four times	2
What this pack is	2
What this pack is NOT	3
The honest overlap picture	3
How to use it	3
 Shared Core — the 10 doctrines all four exams test	 4
D1. Verify before it ships	4
D2. A deterministic mechanism is better than a probabilistic one.	5
D3. Least privilege everywhere	5
D4. Smallest model that passes evals	5
D5. Prefer the simplest pattern that meets the requirement . .	6
D6. Context is a budget	7
D7. Cache-stable prefixes (<i>CCDV + CCAR-P only — out of scope for CCAR-F, absent from CCAO</i>)	7
D8. Batch vs realtime (<i>CCDV + CCAR-P</i>)	8
D9. MCP mental model (<i>CCDV + CCAR-F + CCAR-P. Concepts only in CCAO’s “connectors”</i>)	8
D10. Enforced vs advisory in Claude Code (<i>CCDV + CCAR-F + CCAR-P</i>)	8
Exam-craft constants (identical across all four)	8
 Shared Core — the 10 doctrines all four exams test	 9
D1. Verify before it ships	9
D2. Deterministic beats probabilistic	9
D3. Least privilege everywhere	10
D4. Smallest model that passes evals	10
D5. Prefer the simplest pattern that meets the requirement . .	11

D6. Context is a budget 11

D7. Cache-stable prefixes (*CCDV + CCAR-P only — out of scope for CCAR-F, absent from CCAO*) 12

D8. Batch vs realtime (*CCDV + CCAR-P*) 12

D9. MCP mental model (*CCDV + CCAR-F + CCAR-P; concepts only in CCAO’s “connectors”*) 12

D10. Enforced vs advisory in Claude Code (*CCDV + CCAR-F + CCAR-P*) 13

Exam-craft constants (identical across all four) 13

Divergence Map — where shared knowledge must NOT cross exams **13**

1. Scope fences (the same topic, in one exam and out of another) 14

2. Terminology deltas (graded against each guide’s own words) 15

3. Same stem, different correct answer (the dangerous ones) . 16

4. Per-exam unique mass (what ONLY that pack teaches — protect time for it) 17

5. Morning-of checklists (20 minutes, that exam’s column only) 18

Divergence Map — where shared knowledge must NOT cross exams **18**

1. Scope fences (the same topic, in one exam and out of another) 18

2. Terminology deltas (graded against each guide’s own words) 20

3. Same stem, different correct answer (the dangerous ones) . 21

4. Per-exam unique mass (what ONLY that pack teaches — protect time for it) 21

5. Morning-of checklists (20 minutes, that exam’s column only) 23

Cross-Cert Synergy Pack — study once, deploy four times

Built for back-to-back exam runs — most useful when sitting two or more of the four certifications within the same week.

What this pack is

The four exams test **ten shared doctrines at four different altitudes**, plus a small set of genuinely shared facts. This pack centralizes those so you learn them once, then maps exactly where the exams **diverge** so shared knowledge never bleeds into the wrong answer sheet.

File	Job
01-shared-core.md	The 10 doctrines + shared facts, each with a four-exam framing table — the ~third of total content you should only learn once
02-divergence-map.md	Anti-contamination: scope fences, terminology deltas, per-exam unique mass, and “same stem, different correct answer” cases

What this pack is NOT

- **Not a replacement** for the four packs — every exam has a large unique mass (CCAO product judgment, CCAR-F Agent SDK mechanics, CCDV Integration detail, CCAR-P soft domains $\approx 35\%$) that only its own pack covers.
- **Not merged content.** Where the exams disagree on scope or vocabulary, the divergence map wins — read it the morning of each exam.

The honest overlap picture

- **Doctrine overlap: high** (the 10 principles in 01 recur in every guide’s sample questions and objectives — this is where the synergy lives).
- **Fact overlap CCDV $\leftrightarrow$ CCAR-P: high** (caching, batch, patterns, model selection — mostly the same technical facts at builder vs architect altitude).
- **Fact overlap CCAR-F $\leftrightarrow$ CCDV: medium, with fences** (agents/tools/Claude Code shared conceptually, but CCAR-F excludes several CCDV core topics).
- **Fact overlap CCAO $\leftrightarrow$ the technical three: low** (model tiers and the verification instinct transfer; almost nothing else does — don’t study API mechanics for CCAO or Projects mechanics for CCDV).

How to use it

1. Day 1: 01-shared-core.md end to end — it’s the spine everything hangs on.

- 2. Then: one focused pass per exam, in exam order, with late-week second passes.
- 3. Every exam morning: 20 minutes on that exam’s column in 02-divergence-map.md — fences and vocabulary, nothing else.

Shared Core — the 10 doctrines all four exams test

Note: This edition follows the ASD-STE100 Simplified Technical English writing rules. Simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, Haiku, Sonnet, Opus, Fable, MCP, CLAUDE.md, prompting, caching, effort, Batch, Pearson VUE) are exceptions and stay as written.

Learn each doctrine once. Then read its four-exam table. The principle is the same. Each exam uses a different level of detail and vocabulary. Sources: the four official Exam Guides v1.0 (July 2026) and their sample questions.

D1. Verify before it ships

Do not trust anything Claude produces until you check it. Open citations. Recompute numbers. Validate outputs. The check must match the blast radius of a wrong answer.

Exam	How the exam tests it
CCAO-F	The largest domain (21%): ship, revise, or reject judgment. Check the citation before the compliance send (official sample)
CCAR-F	Confidence calibration, stratified sampling, claim-source provenance (Domain 5)
CCDV-F	Eval harnesses, regression sets, validators before side effects
CCAR-P	Gold + adversarial + regression eval sets. Shadow before cutover. Code judges before model judges

D2. A deterministic mechanism is better than a probabilistic one.

Prompts and few-shots *lower* failure rates. Gates, hooks, and code make the failure rate *zero*. Stems that contain must / never / compliance / a dollar threshold ask for the deterministic mechanism.

Exam	The mechanism vocabulary
CCAO-F	Human approval gate before external/irreversible sends
CCAR-F	Programmatic prerequisite. PreToolUse-style interception hook (sample Q1)
CCDV-F	Schema validation in code. Claude Code settings/hooks enforce. CLAUDE.md advises
CCAR-P	Workflow gates + authZ enforced in the tool server. Caps in code, never model judgment

D3. Least privilege everywhere

For tools, data, permissions, and sharing: grant the minimum that is sufficient for the task. Removal of capability is a *valid first fix*. It is not an admission of failure.

Exam	Flavor
CCAO-F	Sharing permissions (view vs edit), connector scope, PII minimization
CCAR-F	Subagent tools restriction lists. The reviewer gets read-only
CCDV-F	Tool allowlists. Injection blast-radius control
CCAR-P	Remove tools first (sample Q1). Per-agent credentials. authZ at the owning seam

D4. Smallest model that passes evals

Default to the lightest tier that meets the quality requirement. Escalate only on evidence. “Use the model with the most capability” is a distractor

on all four exams.

Shared facts (learn once — dated Aug 2026, check again in exam week): Tiers run Haiku (fast/cheap, 200K ctx) → Sonnet (default). Then Opus (deep) → Fable-class (longest-horizon frontier). Current 4.6+ generations use **adaptive thinking**. Fixed budget_tokens is legacy. Current tiers remove sampling params like temperature. Effort (low to max, xhigh from Opus 4.7 on) controls cost and depth inside a model. Current frontier and balanced tiers have 1M-class context.

Exam	Altitude
CCAO-F	Product picker by task type. The guide names only Haiku/Sonnet/Opus. Fable-maximalism is a trap
CCAR-F	Cheaper model per subagent (model override) for bulk reading
CCDV-F.	Eval-driven selection + effort setting + fast mode (CCDV-only fact set).
CCAR-P	Eval-proven promotion/demotion. Router architectures. Cost envelopes

D5. Prefer the simplest pattern that meets the requirement

Single call → workflow → agent → multi-agent. Move right only when evidence shows the left option fails. The left options give auditability. The right

Exam	Framing
CCAO-F	Chat → Project → Skill/connector ladder. Do not build a workflow when a good prompt does the job
CCAR-F	Prompt chaining (steps enumerable) vs adaptive decomposition (steps emerge). Agent = model-driven tool choice

Exam	Framing
CCDV-F	Workflow + smaller model beats agent-everywhere. Frameworks add structure. They do not add hidden capability
CCAR-P	The leftmost-pattern rule, verbatim. Multi-agent without role boundaries is not a real design

D6. Context is a budget

Everything you send counts. More context is not more quality. Curate the context. Retrieve it in steps. Protect what you must never summarize away.

Exam	Framing
CCAO-F	Project knowledge hygiene: stale or duplicate files damage every chat
CCAR-F	Lost-in-the-middle, case-facts blocks, per-file + integration passes for attention dilution (sample Q12)
CCDV-F	Context budgeting, compaction, tool-catalog size, token counting
CCAR-P	Progressive discovery vs monolithic dump. Compaction drift as a failure mode

D7. Cache-stable prefixes (CCDV + CCAR-P only – out of scope for CCAR-F, absent from CCAO)

Put stable content first. Put volatile content last. Render order is tools → system → messages. Use about 4 breakpoints. A byte change invalidates everything after it. A thinking-config change invalidates message-level entries. The tools/system prefix survives. Cached tokens still occupy the window. CCAR-P sample Q2 and about half of CCDV cache stems test this one paragraph.

D8. Batch vs realtime (CCDV + CCAR-P)

If a human waits, use sync. Stream if you need better perceived latency. If the work is overnight, bulk, or cost-driven, use **Batch**. Batch is about 50% of sync cost. It is async. It has no streaming. Cache hits are best-effort. It has no pre-warm. Both exams' sample sets start with this trade.

D9. MCP mental model (CCDV + CCAR-F + CCAR-P. Concepts only in CCAO's "connectors")

Know the host, client, and server roles. Know tools, resources, and prompts. Build an MCP server when **reuse across apps or teams** matters. Third-party servers are supply chain. Trust them and set their scope. Mechanics differ per exam (see the divergence map). CCAR-F wants config and error mechanics. CCAR-P wants authZ-in-the-server architecture. CCDV wants the connector and integration picture.

D10. Enforced vs advisory in Claude Code (CCDV + CCAR-F + CCAR-P)

Settings, permissions, managed policy, and hooks **enforce**. CLAUDE.md and prompts **advise**. Org-managed settings cannot be overridden. Version control distributes project-level config. User-level config does not. That difference is a recurring diagnostic stem in all three technical exams.

Exam-craft constants (identical across all four)

- **720 pass on a 100–1,000 scaled score.** This is not a fixed %. The program equates forms.
- **No penalty for wrong answers.** Never leave a blank. Flag the item and return to it.
- **Multiple-response is all-or-nothing.** Each item states how many to select.
- **Retakes:** 14/30/90-day waits. Max 4 attempts per rolling 12 months. This is relevant to your run. You cannot retry a failed exam inside this window's remaining days. Sleep well before the exam. Do not study intensively at the last minute.
- **Constraint-first reading:** Read the question sentence first. Find the stem's constraints. Remove any option that violates one con-

straint. Then select the simplest survivor. This is the shared solving algorithm. Only the vocabulary changes.

Shared Core — the 10 doctrines all four exams test

Learn each doctrine once, then read its four-exam table: same principle, different altitude and vocabulary per exam. Sources: the four official Exam Guides v1.0 (July 2026) and their sample questions.

D1. Verify before it ships

Nothing Claude produces is trusted until checked — citations opened, numbers recomputed, outputs validated — and the check is proportionate to the blast radius of being wrong.

Exam	How it’s tested
CCAO-F	The 21% mega-domain: ship/revise/reject judgment; verify the citation before the compliance send (official sample)
CCAR-F	Confidence calibration, stratified sampling, claim–source provenance (Domain 5)
CCDV-F	Eval harnesses, regression sets, validators before side effects
CCAR-P	Gold + adversarial + regression eval sets; shadow before cutover; code judges before model judges

D2. Deterministic beats probabilistic

Prompts and few-shots *lower* failure rates; gates, hooks, and code make them zero. Stems containing must / never / compliance / a dollar threshold are asking for the deterministic mechanism.

Exam	The mechanism vocabulary
CCAO-F	Human approval gate before external/irreversible sends
CCAR-F	Programmatic prerequisite; PreToolUse-style interception hook (sample Q1)
CCDV-F	Schema validation in code; Claude Code settings/hooks enforce, CLAUDE.md advises
CCAR-P	Workflow gates + authZ enforced in the tool server; caps in code, never model judgment

D3. Least privilege everywhere

Tools, data, permissions, sharing: grant the minimum that does the job, and removing capability is a *first-class fix*, not an admission of failure.

Exam	Flavor
CCAO-F	Sharing permissions (view vs edit), connector scope, PII minimization
CCAR-F	Subagent tools restriction lists; reviewer gets read-only
CCDV-F	Tool allowlists; injection blast-radius control
CCAR-P	Remove tools first (sample Q1); per-agent credentials; authZ at the owning seam

D4. Smallest model that passes evals

Default to the lightest tier that meets the quality bar; escalate only on evidence; “use the biggest model” is a planted distractor on all four exams.

Shared facts (learn once — dated Aug 2026, re-verify exam week): tiers run Haiku (fast/cheap, 200K ctx) → Sonnet (default) → Opus (deep) → Fable-class (longest-horizon frontier); current 4.6+ generations use **adaptive thinking** (fixed budget_tokens is legacy; sampling params like temperature are removed on current tiers); **effort** (low→max, xhigh

from Opus 4.7 on) is the intra-model cost/depth dial; 1M-class context on current frontier/balanced tiers.

Exam	Altitude
CCAO-F	Product picker by task type — guide names only Haiku/Sonnet/Opus;
CCAR-F	Fable-maximalism is a trap Cheaper model per subagent (model override) for bulk reading
CCDV-F	Eval-driven selection + effort dial + fast mode (CCDV-only fact set)
CCAR-P	Eval-proven promotion/demotion; router architectures; cost envelopes

D5. Prefer the simplest pattern that meets the requirement

Single call → workflow → agent → multi-agent: move right only when evidence says the left fails. Auditability lives left; flexibility lives right.

Exam	Framing
CCAO-F	Chat → Project → Skill/connector ladder; don't build a workflow where a good prompt does
CCAR-F	Prompt chaining (steps enumerable) vs adaptive decomposition (steps emerge); agent = model-driven tool choice
CCDV-F	Workflow + smaller model beats agent-everywhere; frameworks add structure, not magic
CCAR-P	The leftmost-pattern rule verbatim; multi-agent without role boundaries is theater

D6. Context is a budget

Everything sent counts; more context is not more quality. Curate, retrieve progressively, and protect what must never be summarized away.

Exam	Framing
CCAO-F	Project knowledge hygiene: stale/duplicate files poison every chat
CCAR-F	Lost-in-the-middle, case-facts blocks, per-file + integration passes for attention dilution (sample Q12)
CCDV-F	Context budgeting, compaction, tool-catalog size, token counting
CCAR-P	Progressive discovery vs monolithic dump; compaction drift as failure mode

D7. Cache-stable prefixes (CCDV + CCAR-P only — out of scope for CCAR-F, absent from CCAO)

Stable content first, volatile last; render order tools → system → messages; ~4 breakpoints; byte changes invalidate everything after them; thinking-config changes invalidate message-level entries (tools/system prefix survives); cached tokens still occupy the window. CCAR-P sample Q2 and half of CCDV's cache stems are this one paragraph.

D8. Batch vs realtime (CCDV + CCAR-P)

Human waiting → sync (stream for perceived latency). Overnight/bulk/cost → **Batch** (~50% cost, async, no streaming, best-effort cache hits, no pre-warm). Both exams' sample sets open with this trade.

D9. MCP mental model (CCDV + CCAR-F + CCAR-P; concepts only in CCAO's "connectors")

Host/client/server roles; tools/resources/prompts; build an MCP server when **reuse across apps/teams** matters; third-party servers are supply chain — trust and scope them. Mechanics diverge per exam (see divergence map): CCAR-F wants config/error mechanics, CCAR-P wants authZ-in-the-server architecture, CCDV wants the connector/integration picture.

D10. Enforced vs advisory in Claude Code (CCDV + CCAR-F + CCAR-P)

Settings, permissions, managed policy, and hooks **enforce**; CLAUDE.md and prompts **advise**. Org-managed settings are non-overridable; project-level config travels via version control, user-level config doesn't — that difference is a recurring diagnostic stem in all three technical exams.

Exam-craft constants (identical across all four)

- **720 pass on a 100–1,000 scaled score** — not a fixed %; forms are equated.
- **No penalty for wrong answers** — never leave a blank; flag and return.
- **Multiple-response is all-or-nothing**; each item states how many to select.
- **Retakes**: 14/30/90-day waits, max 4 attempts per rolling 12 months — relevant to your run: a failed exam cannot be retried inside this window's remaining days, so protect sleep over cramming.
- **Constraint-first reading**: question sentence first, harvest the stem's constraints, kill options that violate any one, then pick the simplest survivor. This is the shared solving algorithm; only the vocabulary changes.

Divergence Map — where shared knowledge must NOT cross exams

Note: This edition follows the ASD-STE100 Simplified Technical English writing rules. Simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, Claude Code, MCP, Task, Memory, prompting, caching, Batch, fast mode, RAG, Haiku, Sonnet, Opus, Fable) are exceptions and stay as written.

This file stops you from using knowledge from the wrong exam. Read the relevant exam's fences for 20 minutes each exam morning. When this file and your instinct disagree, follow this file. Each exam uses its own guide's scope and vocabulary.

1. Scope fences (the same topic, in one exam and out of another)

Topic	CCAO-F	CCAR-F	CCDV-F	CCAR-P
Prompt-caching mechanics	absent	explicitly OUT of scope	core (Integration 33.1%)	core (sample Q2)
Cloud providers (Bedrock/Vertex/Foundry)	absent	explicitly OUT	in (deployment surfaces, hosting)	in (CSP route governance, file 02 D5)
Streaming / SSE vs websockets	absent	explicitly OUT	in (module 06)	light (latency lever)
Pricing / rate limits	absent	explicitly OUT	concepts (cost levers, batch 50%)	cost envelopes, no price sheets
Agent SDK specifics (Task tool, AgentDefinition, hooks, fork_session)	absent	CCAR-F ONLY — 27% domain	generic agent loops only	pattern-level only
claude.ai product (Projects, Artifacts, Memory, Research, incognito)	core — the whole exam	absent	absent	absent
Fast mode	absent	out (pricing-adjacent)	named in the guide (MSO)	not named
Embeddings / vector DBs	absent	explicitly OUT	light (RAG context)	in (RAG architecture)
Vision / multimodal	absent	explicitly OUT	in (API mechanics)	light (multimodal design)

Topic	CCAO-F	CCAR-F	CCDV-F	CCAR-P
GTM / stakeholder / lifecycle	absent	absent	absent	CCAR-P ONLY — ~21%
4D AI Fluency framework	in (Delegation, Description, Discernment, Diligence)	absent	absent	absent
Eval harness engineering	judgment-level only	calibration/sampling flavor	light (2.6%)	deep — 16%

Usage rule: An answer option that uses a topic marked OUT for that day’s exam is a distractor by definition. Remove it without extra thought. This is especially useful on CCAR-F.

2. Terminology deltas (graded against each guide’s own words)

Concept	Say this per exam
Subagent spawning tool	CCAR-F: “Task” tool (the guide’s vocabulary — know the current-docs rename to Agent. Answer with Task)
Value pillars	CCAR-P: efficiency, transformation, productivity, cost, performance SLAs — the guide’s exact five
Model ladder	CCAO-F: Haiku / Sonnet / Opus — the guide names three tiers only. Fable appears in options as an over-buy trap
Prompting technique names	CCDV-F: zero-shot / single-shot / multi-shot defined exactly. CCAO-F uses plain-language framing

Concept	Say this per exam
"Memory"	CCAO-F: the claude.ai Memory feature (entries, controls, incognito). CCAR-F/CCDV: Claude Code auto memory / /memory. These are different products. Never mix them

3. Same stem, different correct answer (the dangerous ones)

"The agent skipped a required compliance step." - CCAR-F → programmatic prerequisite or PreToolUse-style interception hook. - CCAR-P → workflow gate + authZ enforced in the tool server (architecture altitude). - CCAO-F → human approval gate before the action goes outside. Same doctrine (D2). The graded answer names *that exam's* mechanism.

"Improve extraction reliability." - CCDV-F → JSON schema + strict validation + few-shot anchors. - CCAR-F → retry-with-error-feedback loops, nullable fields to prevent fabrication. - CCAR-P → validators in code before side effects, inside a workflow stage. (And never "lower temperature". That control is gone on current tiers. All packs now state this fact.)

"Which model should they use?" - CCAO-F → task-type picker (high-volume simple → Haiku-class. Deep → Opus-class). - CCDV-F → smallest that passes evals + effort setting. Use fast mode only if latency is critical AND the premium is justified. - CCAR-P → eval-proven promotion with router/canary architecture around it.

"Claude keeps getting things wrong for the team." - CCAO-F → Project knowledge hygiene / Memory entry cleanup / configuration first. - CCDV-F / CCAR-F → project-level vs user-level Claude Code config. Enforce vs advise. - CCAR-P → Diagnose with evals first (retrieval before model — sample Q3). Then check config.

"Handle this long/complex conversation context." - CCAO-F → start a fresh chat with a summary. Use Project knowledge for durable facts. - CCAR-F → scratchpad files, case-facts blocks, fresh-session-with-summary vs -resume. - CCDV-F / CCAR-P → compaction/context editing features + cache-stable design.

4. Per-exam unique mass (what **ONLY** that pack teaches — protect time for it)

Exam	Unique, non-transferable content	Where
CCAO-F	Product feature judgment (Projects/Artifacts/Memory/Re-search/incognito/code execution), plan facts, 4D fluency, high-risk-use governance	whole pack, esp. 03/04/06
CCAR-F	Agent SDK mechanics (stop_reason loop anti-patterns, AgentDefinition, allowedTools, hooks, fork_session). Claude Code config mechanics (@import, rules globs, SKILL.md keys, -output-format json). Also the 6 published scenarios	08, 09, 04/05 supplements
CCDV-F	Integration API detail (streaming, caching mechanics, batch mechanics, schemas, pinning), fast mode, LLM fundamentals, frameworks (Strands/Lang-Graph/PydanticAI), SDLC/websockets	03, 01, 06

Exam	Unique, non-transferable content	Where
CCAR-P	Soft domains (~35%): stakeholder/GTM, life-cycle/hypercare/DRI, enablement. CSP route governance. Judge calibration. Review-routing matrix	02 D5/M7, 03 D4, 04, 05

5. Morning-of checklists (20 minutes, that exam’s column only)

- **Before CCAO-F:** fences (the exam does not grade API content). Haiku/Sonnet/Opus framing. Check → gate → own. Memory = claude.ai feature today.
- **Before CCAR-F:** the OUT list removes distractors. Use “Task” vocabulary. Deterministic > probabilistic. Attribute the failure to the coordinator, not to the subagent. Review the 6 scenarios.
- **Before CCDV-F:** caching, batch, and fast-mode facts are in scope. The constraint keyword decides. Check the freshness watchlist once.
- **Before CCAR-P:** everything technical is in scope PLUS the soft 35%. Mechanism + metric + owner. Leftmost pattern. Official pillar wording. Constraint-elimination drill at 1.9 min/item.

Divergence Map — where shared knowledge must NOT cross exams

The anti-contamination file. Read the relevant exam’s fences for 20 minutes each exam morning. When this file and instinct disagree, this file wins — each exam is graded against its own guide’s scope and vocabulary.

1. Scope fences (the same topic, in one exam and out of another)

Topic	CCAO-F	CCAR-F	CCDV-F	CCAR-P
Prompt-caching mechanics	absent	explicitly OUT of scope	core (Integration 33.1%)	core (sample Q2)
Cloud providers (Bedrock/Vertex/Foundry)	absent	explicitly OUT	in (deployment surfaces, hosting)	in (CSP route governance, file 02 D5)
Streaming / SSE vs websockets	absent	explicitly OUT	in (module 06)	light (latency lever)
Pricing / rate limits	absent	explicitly OUT	concepts (cost levers, batch 50%)	cost envelopes, no price sheets
Agent SDK specifics (Task tool, AgentDefinition, hooks, fork_session)	absent	CCAR-F ONLY — 27% domain	generic agent loops only	pattern-level only
claude.ai product (Projects, Artifacts, Memory, Research, incognito)	core — the whole exam	absent	absent	absent
Fast mode	absent	out (pricing-adjacent)	named in the guide (MSO)	not named
Embeddings / vector DBs	absent	explicitly OUT	light (RAG context)	in (RAG architecture)
Vision / multimodal	absent	explicitly OUT	in (API mechanics)	light (multimodal design)
GTM / stakeholder / lifecycle	absent	absent	absent	CCAR-P ONLY — ~21%

Topic	CCAO-F	CCAR-F	CCDV-F	CCAR-P
4D AI Fluency framework	in (Delegation, Description, Discernment, Diligence)	absent	absent	absent
Eval harness engineering	judgment-level only	calibration/sampling flavor	light (2.6%)	deep — 16%

Usage rule: an answer option resting on a topic marked OUT for that day’s exam is a distractor by definition — kill it without deliberation (this is a weapon on CCAR-F especially).

2. Terminology deltas (graded against each guide’s own words)

Concept	Say this per exam
Subagent spawning tool	CCAR-F: “Task” tool (the guide’s vocabulary — know the current-docs rename to Agent, answer with Task)
Value pillars	CCAR-P: efficiency, transformation, productivity, cost, performance SLAs — the guide’s exact five
Model ladder	CCAO-F: Haiku / Sonnet / Opus — the guide names three tiers only; Fable appears in options as an over-buy trap
Prompting technique names	CCDV-F: zero-shot / single-shot / multi-shot defined exactly; CCAO-F uses plain-language framing
“Memory”	CCAO-F: the claude.ai Memory feature (entries, controls, incognito). CCAR-F/CCDV: Claude Code auto memory / /memory. Different products — never mix

3. Same stem, different correct answer (the dangerous ones)

“The agent skipped a required compliance step.” - CCAR-F → programmatic prerequisite or PreToolUse-style interception hook. - CCAR-P → workflow gate + authZ enforced in the tool server (architecture altitude). - CCAO-F → human approval gate before the action leaves the building. Same doctrine (D2); the graded answer names *that exam’s* mechanism.

“Improve extraction reliability.” - CCDV-F → JSON schema + strict validation + few-shot anchors. - CCAR-F → retry-with-error-feedback loops, nullable fields to prevent fabrication. - CCAR-P → validators in code before side effects, inside a workflow stage. (And never “lower temperature” — gone on current tiers, all packs corrected.)

“Which model should they use?” - CCAO-F → task-type picker (high-volume simple → Haiku-class; deep → Opus-class). - CCDV-F → smallest that passes evals + effort dial; fast mode only if latency-critical AND premium justified. - CCAR-P → eval-proven promotion with router/canary architecture around it.

“Claude keeps getting things wrong for the team.” - CCAO-F → Project knowledge hygiene / Memory entry cleanup / configuration first. - CCDV-F / CCAR-F → project-level vs user-level Claude Code config; enforce vs advise. - CCAR-P → eval-diagnose first (retrieval before model — sample Q3), then config.

“Handle this long/complex conversation context.” - CCAO-F → start a fresh chat with a summary; Project knowledge for durable facts. - CCAR-F → scratchpad files, case-facts blocks, fresh-session-with-summary vs —resume. - CCDV-F / CCAR-P → compaction/context editing features + cache-stable design.

4. Per-exam unique mass (what ONLY that pack teaches — protect time for it)

Exam	Unique, non-transferable content	Where
CCAO-F	Product feature judgment (Projects/Artifacts/Memory/Research/incognito/code execution), plan facts, 4D fluency, high-risk-use governance	whole pack, esp. 03/04/06
CCAR-F	Agent SDK mechanics (stop_reason loop anti-patterns, AgentDefinition, allowedTools, hooks, fork_session), Claude Code config mechanics (@import, rules globs, SKILL.md keys, -output-format json), the 6 published scenarios	08, 09, 04/05 supplements
CCDV-F	Integration API detail (streaming, caching mechanics, batch mechanics, schemas, pinning), fast mode, LLM fundamentals, frameworks (Strands/Lang-Graph/PydanticAI), SDLC/websockets	03, 01, 06
CCAR-P	Soft domains (~35%): stakeholder/GTM, life-cycle/hypercare/DRI, enablement; CSP route governance; judge calibration; review-routing matrix	02 D5/M7, 03 D4, 04, 05

5. Morning-of checklists (20 minutes, that exam's column only)

- **Before CCAO-F:** fences (no API content will be graded); Haiku/Sonnet/Opus framing; verify → gate → own; Memory = claude.ai feature today.
- **Before CCAR-F:** the OUT list is your distractor-killer; “Task” vocabulary; deterministic > probabilistic; blame the coordinator; walk the 6 scenarios.
- **Before CCDV-F:** caching/batch/fast-mode facts are in scope; the constraint keyword decides; check the freshness watchlist once.
- **Before CCAR-P:** everything technical is in scope PLUS the soft 35% — mechanism + metric + owner; leftmost pattern; official pillar wording; constraint-elimination drill at 1.9 min/item.